

PaperPass[免费版]查重报告

简明打印版

查重结果(相似度):

总 体: 2%

本地库: 2% (本地库包含学术联合库、期刊库、学位库、会议库、共享联合库)

互联网: (免费版不检测互联网资源)

检测版本: 免费版(仅检测中文)

报告编号: D3VJ6A07886F103D1

论文题目: verilog-i2c-thesis_AIGC_down

论文作者: 佚名

论文字数: 25672

段落个数: 175

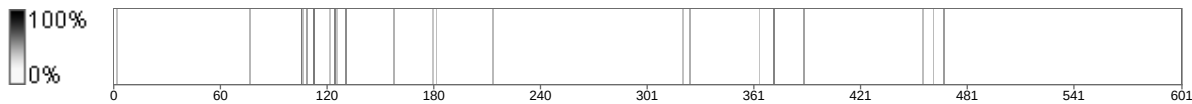
句子个数: 601

提交时间: 2026-5-16 4:56:15

比对范围: 学术联合库、期刊库、硕博学位库、会议库、共享联合库

查询真伪: <https://www.paperpass.com/check>

句子相似度分布图:



本地库相似资源列表(学术联合库、期刊库、硕博学位库、会议库、共享联合库):

没有找到与本地库相似度高的资源

基于 Verilog 的 I2C 接口组件库设计与分析

摘要

随着 **查重 41%** 在嵌入式控制、数据采集、工业通信和 SoC 原型验证中的使用越来越多，**外设接口能否复用，已经成为数字系统开发中需要认真考虑的问题**。I2C 总线只占用 SDA 和 SCL 两根主要信号线，协议层次也比较清楚，很多 EEPROM、时钟芯片、传感器、电源管理芯片以及低速控制器件都支持这一接口。对于 FPGA 项目来说，如果每次都重新编写一套 I2C 通信逻辑，开发时间会被拉长，时序控制、应答判断、总线占用、时钟拉伸和异常处理等细节也容易反复出错。基于这一背景，整理并分析一套结构清楚、便于复用、能够接入片上总线且带有验证支撑的 I2C 接口组件库，对工程实践和本科阶段的学习都有一定价值。

本文选择 `verilog-i2c-master` 作为分析对象，重点围绕这个 Verilog I2C 接口组件库的功能组成、模块关系、实现思路和测试支撑进行整理。项目中包含 I2C 主控制器、I2C 从控制器、AXI-Lite 与 Wishbone 相关包装模块、I2C 到片上总线的桥接逻辑、上电初始化模块、单寄存器从设备示例，以及基于 MyHDL 的协同仿真测试平台。论文先从 FPGA 外设控制场景切入，说明 I2C 接口组件库需要解决的主要问题；再结合项目目录和模块划分，分析其“协议核心、总线适配、应用示例、验证支撑”几类内容之间的关系；随后对 `i2c_master.v`、`i2c_slave.v`、`i2c_master_axil.v`、`i2c_slave_axil_master.v`、`i2c_init.v` 等文件进行重点说明，讨论它们的接口、数据流和设计侧重点；最后根据现有测试材料，对验证方式、材料能够支撑的结论以及后续可完善的方向进行归纳。

分析结果可以看出，`verilog-i2c-master` 在模块拆分、参数配置和工程复用方面表现较完整。项目的主从协议模块使用 AXI Stream 风格接口把上层控制与底层 I2C 时序分开处理，AXI-Lite 和 Wishbone 包装层又进一步降低了与片上系统集成难度。测试部分通过 Python/MyHDL 行为模型配合 Icarus Verilog 仿真，围绕主机、从机、初始化流程和桥接模块安排了验证设计。不过，现有材料也明确说明该仓库已经停止维护，同时没有给出综合、时序、资源占用和板级运行结果。因此，本文更倾向于把它作为 FPGA I2C 接口设计的学习样例、二次移植基础和 HDL 模块化设计案例，而不是直接认定为可以不经验证投入生产环境的最终 IP。

关键词：I2C 总线，Verilog，FPGA，AXI-Lite，Wishbone，协同仿真

Abstract

With the increasing use of FPGA devices in embedded control, data acquisition, industrial communication, and system-on-chip prototyping, reusable peripheral interface modules have become important in digital system development. The I2C bus is widely used for connecting EEPROMs, clock devices, sensors, power-management chips, and low-speed control peripherals because it requires few pins and has a relatively simple protocol structure. In FPGA systems, repeatedly developing I2C communication logic for each project may increase development time and introduce errors in timing control, acknowledge detection, clock stretching, and exception handling. Therefore, it is meaningful to study a reusable I2C interface component library that is modular, parameterized, easy to integrate with on-chip buses, and supported by a verification environment.

This thesis studies the `verilog-i2c-master` project and analyzes its Verilog-based I2C interface component library in terms of functional requirements, system architecture, module design, implementation logic, and verification methods. The project provides an I2C master controller, an I2C slave controller, AXI-Lite and Wishbone wrappers, I2C-to-on-chip-bus bridge modules, an initialization state-machine template, a single-register I2C slave example, and a MyHDL-based co-simulation testbench. This thesis first introduces the application background of I2C communication in FPGA peripheral control, and then analyzes the project requirements and design constraints. It further explains the layered architecture of the project, including the core protocol engines, bus wrappers, application modules, and verification support. The core modules such as `i2c_master.v`, `i2c_slave.v`, `i2c_master_axil.v`, `i2c_slave_axil_master.v`, and `i2c_init.v` are discussed in detail. Finally, the verification design and limitations of the project are summarized.

The analysis shows that the project has good modularity, parameterization, and reusability. The core modules use AXI Stream style interfaces to decouple protocol timing from control logic, while the AXI-Lite and Wishbone wrappers make the design suitable for common FPGA-based systems. The verification environment combines Python/MyHDL bus models with Icarus Verilog co-simulation. However, the available materials also indicate that the repository has been deprecated and lacks synthesis reports, timing reports, board-level validation results, and resource utilization data. Therefore, this thesis regards the project as a useful learning reference and reusable design basis rather than a production-ready solution without further validation.

Key words: I2C bus, Verilog, FPGA, AXI-Lite, Wishbone, co-simulation

1 绪论

1.1 研究背景

在嵌入式系统和数字逻辑系统中，FPGA 一方面可以完成高速数据处理、协议转换和实时控制等工作，另一方面也常常要和板上的低速外设通信。与高速串行接口或复杂并行总线相比，I2C 的连接线更少，硬件连接方式相对简单，而且很多外围芯片都提供 I2C 接口，所以它经常出现在芯片配置、传感器读取、时钟控制、电源管理以及寄存器访问等场景中。对于一个 FPGA 系统而言，I2C 逻辑未必是最核心的计算模块，但系统启动、外设初始化和运行状态管理往往离不开它。

从协议概念上看，I2C 并不复杂，但真正写成 HDL 模块时，要处理的细节并不少。主机侧要负责开始和停止条件、地址传输、读写控制以及 ACK/NACK 判断；从机侧则要处理地址匹配、应答、接收、发送和必要的时钟拉伸。若系统还希望通过 AXI-Lite 或 Wishbone 这类片上总线来控制 I2C 外设，就需要再增加寄存器映射、FIFO 缓冲、命令通道和状态反馈等内容。没有可复用组件时，这些逻辑很容易在多个项目中被反复编写，既影响效率，也会让验证工作变得零散。

`verilog-i2c-master` 项目正是面向这一类需求的 Verilog I2C 接口组件库。根据项目报告，该项目提供了一组可复用的 I2C 主从控制器、片上总线包装层、初始化状态机和测试平台，目标是在 Verilog 设计中快速集成 I2C 通信能力。该项目既包含核心协议模块，也包含 AXI-Lite 和 Wishbone 适配层，还配套了 Python/MyHDL

行为模型与 Icarus Verilog 协同仿真环境。这种组织方式使其不仅可作为工程复用模块，也适合作为本科阶段学习 HDL 模块化设计、总线接口封装和数字系统验证方法的案例。

1.2 研究目的与意义

本文的研究目的在于对 `verilog-i2c-master` 项目进行系统化论文整理，从本科毕业论文的角度说明该项目的设计背景、需求分析、总体架构、模块划分、实现逻辑和验证方式。本文并非重新从零实现一个 I2C 控制器，而是在项目报告所总结的事实基础上，将项目内容转化为符合毕业论文表达习惯的技术文档，使项目的工程价值、设计思路和不足之处得到较完整呈现。

从工程学习的角度来看，本文的价值主要可以从几个方面理解。一方面，I2C 控制器属于 FPGA 系统中常见的基础接口，分析该项目有助于理解一个位级协议控制器怎样进一步封装成可复用的片上总线外设。另一方面，项目同时给出了 AXI-Lite 和 Wishbone 相关接口，能够体现总线适配和模块复用 in HDL 工程中的作用。除此之外，项目配套的 MyHDL 协同仿真平台也说明，HDL 模块验证不能只停留在编译通过，而要围绕协议行为、数据流向和边界情况设计测试激励。

从学习层面看，该项目比较适合用来观察 FPGA 外设接口组件库是怎样组织起来的。一个真正便于复用的 HDL 工程，一般不会只放一个 RTL 文件，还需要有明确的接口说明、可调整的参数、面向系统集成的包装层、必要的应用示例和测试环境。`verilog-i2c-master` 的目录安排与模块划分基本体现了这些特点。把它整理成论文形式后，可以帮助读者把协议功能、系统集成和验证支撑联系起来理解。

查重 45%

1.3 国内外研究与应用概况

I2C 总线最早由飞利浦公司提出，主要面向低速片间通信。它通常只需要 SDA 和 SCL 两条信号线，不同从设备可以依靠地址进行区分，因此在嵌入式硬件平台中使用频率较高。在 MCU 应用中，I2C 控制器往往由芯片内部外设提供，开发者通过寄存器或驱动库完成访问；而在 FPGA 应用中，I2C 控制器可能需要根据系统需求自行实现或集成第三方 HDL IP。

在实际 FPGA 项目中，I2C 控制器大致可以分成两类。一类是面向固定外设的专用控制器，初始化序列和访问流程通常直接写在状态机里，适合需求很明确的小型设计；另一类是通用型控制器，它会提供命令、数据、状态反馈和参数配置接口，让 CPU、软核处理器或其他逻辑模块按需要发起访问。`verilog-i2c-master` 更接近第二类，它除了实现基本主从协议，还把控制接口接入 AXI-Lite、Wishbone 等片上总线环境，因此复用范围更宽。

在验证方面，很多 HDL 项目会先使用 Verilog testbench 做仿真，但协议交互一旦变复杂，单纯编写 Verilog 激励的工作量就会增加。Python/MyHDL、cocotb 以及 SystemVerilog/UVM 等方案，为测试组织提供了更多选择。根据项目材料，`verilog-i2c-master` 采用 MyHDL 行为模型配合 Icarus Verilog 进行协同仿真，其中 Python 侧负责生成测试激励和总线模型，Verilog 侧实现被测模块。这种安排更便于描述复杂总线交互，也方便在脚本中加入断言和结果检查。

1.4 研究内容

本文围绕 `verilog-i2c-master` 项目展开，主要研究内容如下。

第一，分析项目背景与需求。本文根据项目报告总结该项目的应用场景，说明其面向 FPGA 外设控制、上电初始化、片上总线与外部微控制器桥接、寄存器访问和数据交互等需求。

第二，分析系统总体架构。项目采用“核心 I2C 协议引擎 + 片上总线包装层 + 应用层示例 + 测试平台”的结构。本文将从模块层次、目录结构和数据流角度说明该架构如何支持复用和扩展。

第三，分析核心模块设计。本文重点讨论 I2C 主控制器、I2C 从控制器、AXI-Lite 主机包装层、Wishbone 主机包装层、I2C 从设备桥接层、初始化模块、单寄存器从设备和 AXI Stream FIFO 等模块的功能定位与接口特点。

第四，分析实现与验证。本文根据项目报告说明其关键实现逻辑，包括协议状态机、流式接口解耦、寄存器映射、总线桥接、初始化脚本化执行和协同仿真测试方式。同时，本文谨慎区分“测试平台存在”和“实际测试结果已运行通过”两类结论，避免把未实际执行的仿真结果写成事实。

第五，总结项目不足和改进方向。项目报告明确指出该仓库已停止维护，且缺少综合结果、时序收敛报告、资源利用率报告和特定 FPGA 板卡工程。因此，本文将在最后提出后续改进建议。

查重 65%

1.5 论文组织结构

查重 42%

本文按照七个章节展开。第一章说明课题背景、研究意义、应用情况、主要内容以及论文安排；第二章介绍后续分析会用到的基础知识，包括 I2C 总线、Verilog HDL、FPGA 模块化设计、AXI-Lite、Wishbone、AXI Stream 和协同仿真；第三章从功能、非功能要求和设计约束三个角度分析项目需求；第四章讨论工程整体结构、模块层次、接口关系与数据流程；第五章进一步分析核心模块的功能定位和实现逻辑；第六章围绕测试平台、验证方法和现有材料能够支撑的结论进行说明；第七章对全文进行总结，并给出后续可以完善的方向。

查重 51%

查重 63%

2 相关技术与理论基础

2.1 I2C 总线协议概述

I2C 属于同步串行通信总线，典型连接由 SCL 时钟线和 SDA 数据线组成。总线设备一般采用开漏或类似三态释放的连接方式，高电平由上拉电阻提供，设备需要表达低电平时才主动拉低信号线。也正因为这种电气特性，多个器件可以挂在同一组信号线上；不过在 HDL 设计中，控制器输出高电平时不能简单按普通推挽信号处理，而应释放总线，让外部上拉或等效电路把线路恢复到高电平。

一次 I2C 访问通常由主设备启动。主设备负责给出 SCL 时钟，并在 SDA 上依次组织开始条件、从设备地址、读写方向、数据字节以及停止条件。从设备收到地址后判断是否需要响应，并在字节传输结束后给出 ACK 或 NACK。读事务中，数据由从设备在主设备提供的时钟节拍下送出；写事务中，数据由主设备发出，从设备完成接收并给出应答。

查重 41%

在 HDL 实现中, I2C 的关键在于正确生成和检测时序。在 I2C 时序中, 开始和停止条件都依赖 SCL 的变化方向: SDA 由高转低表示开始, SDA 由低转高表示停止。普通数据位的变化则通常安排在 SCL 为低的阶段, 等 SCL 处于高电平时保持稳定, 便于接收端采样。控制器还需要处理 ACK 检测、重复起始、连续读写、时钟拉伸和总线忙状态等情况。项目报告中指出, `i2c_master.v` 支持读、写、连续写、起始/重复起始、停止等操作, 并通过 `missed_ack` 状态输出检测从机未应答; `i2c_slave.v` 支持 I2C 读写事务解析, 并能在读取数据尚未准备好时通过保持 SCL 为低实现时钟拉伸。这些功能均与 I2C 协议的典型要求一致。

查重 58%

2.2 Verilog HDL 与 FPGA 设计

Verilog HDL 是数字电路设计中常见的硬件描述语言, 可以描述组合逻辑、时序逻辑、状态机、模块端口以及多层次电路结构。FPGA 开发一般以 Verilog 或 VHDL 这类 RTL 描述作为输入, 再由综合工具把设计映射到目标芯片中的查找表、触发器、块 RAM、DSP 和 I/O 等资源上。

与软件程序不同, HDL 描述的是硬件并行结构。因此, 在设计接口控制器时, 需要关注时钟域、复位逻辑、状态跳转、信号握手、总线时序和资源消耗等问题。I2C 控制器虽然速率相对较低, 但仍需要保证状态机在不同输入条件下稳定运行。项目使用 Verilog 2001 编写, 有利于在常见 FPGA 工具链中复用。项目报告还指出, 多个模块支持参数化配置, 例如数据宽度、地址宽度、FIFO 深度、输入滤波长度和分频参数等, 这体现了 HDL 组件库设计中常见的可配置思想。

2.3 模块化与参数化设计

在 HDL 工程中, 模块化是实现复用的基本前提。对于一个接口组件库来说, 可以把协议处理、数据缓存、寄存器访问、总线桥接和测试支撑拆成相对独立的模块, 让每一部分都有明确边界。这样做的好处是, 单个模块可以独立阅读、仿真和移植。参数化设计则进一步提高了适配能力, 用户可以通过参数改变位宽、深度、地址范围或功能选项, 而不必为相近场景重复编写多份代码。

`verilog-i2c-master` 的模块划分体现了这种思想。`i2c_master.v` 和 `i2c_slave.v` 是协议核心; `i2c_master_axil.v`、`i2c_master_wbs_8.v` 和 `i2c_master_wbs_16.v` 是主机包装层; `i2c_slave_axil_master.v` 和 `i2c_slave_wbm.v` 是从机桥接层; `i2c_init.v` 和 `i2c_single_reg.v` 面向具体应用场景; `axis_fifo.v` 则作为公共缓冲模块服务于数据流解耦。这样的划分使项目既能在简单设计中直接使用核心模块, 也能在片上总线系统中通过包装层进行集成。

2.4 AXI Stream、AXI-Lite 与 Wishbone 总线

AXI Stream 是一种面向流式数据传输的接口风格, 常用于模块之间传递连续数据或命令。其核心思想是通过 `valid` 和 `ready` 信号进行握手, 发送方在数据有效时拉高 `valid`, 接收方在可以接收时拉高 `ready`, 当二者同时有效时完成一次传输。在 I2C 控制器中, AXI Stream 风格接口可以用于输入命令、写入数据和输出读回数据, 从而使上层逻辑不必直接关心 I2C 位级时序。

AXI-Lite 可以理解为 AXI 体系中面向控制寄存器访问的简化接口, 适合传输量不大、以配置和状态读取为主的外设。在 FPGA 或 SoC 系统里, 软核 CPU、硬核处理器或片上控制逻辑都可以通过 AXI-Lite 读写外设寄存器。项目中的 `i2c_master_axil.v` 正是把 I2C 控制器封装成 AXI-Lite 从设备, 并提供状态、命令、数据和分频等寄存器, 使上层软件能够用寄存器访问方式控制 I2C 总线。

Wishbone 是另一种常见开放片上总线规范, 在许多开源软核和 FPGA 项目中使用。项目提供 `i2c_master_wbs_8.v` 和 `i2c_master_wbs_16.v` 两种 Wishbone 主机包装层, 分别适配不同数据位宽的系统。项目还提供 `i2c_slave_wbm.v`, 使外部 I2C 主设备能够间接访问 FPGA 内部 Wishbone 总线。这说明项目不仅关注单一总线环境, 而是希望在多种开源硬件系统中复用。

2.5 协同仿真与测试平台

HDL 设计验证是数字系统开发中的关键环节。对于协议控制器而言, 仅依靠编译通过不能说明功能正确, 必须构造读写事务、边界情况、异常响应和时序变化等测试场景。协同仿真是一种常用方法, 它允许用高级语言编写测试激励和行为模型, 再与 Verilog 被测设计共同运行。

项目报告显示, `verilog-i2c-master` 使用 Python/MyHDL 行为模型和 Icarus Verilog 进行协同仿真。测试目录中包含 `axil.py`、`axis_ep.py`、`i2c.py` 和 `wb.py` 等总线模型文件, 也包含多个 `test_*.py` 和对应 Verilog 测试顶层。Python 负责测试激励、总线行为和断言检查, Verilog 负责实际 DUT 实现。这种模式相比纯 Verilog testbench 更容易组织复杂测试逻辑, 也便于在测试脚本中使用数据结构和流程控制。

3 系统需求分析

3.1 项目目标分析

从项目材料来看, `verilog-i2c-master` 的核心目标是提供一组能够复用的 I2C 主从控制模块, 并配套总线包装层和初始化模块, 让用户在 Verilog 设计中更方便地加入 I2C 通信能力。这个目标可以分成三个方面理解: 其一是协议层, 负责实现 I2C 主模式和从模式的基本访问; 其二是系统集成层, 把协议核心包装成 AXI-Lite、Wishbone 等片上总线可访问的外设; 其三是工程辅助层, 通过初始化模板、简单从设备示例和测试平台, 降低学习、移植和验证成本。

该项目的应用场景包括 FPGA 外设控制、上电初始化、片上总线与外部微控制器桥接、基于 I2C 的寄存器访问和数据交互。对于需要通过 FPGA 配置时钟芯片、传感器、电源管理芯片或其他低速外设的系统而言, I2C 主控制器是基础组件。对于需要让外部 MCU 通过 I2C 控制 FPGA 内部寄存器或片上总线资源的系统而言, I2C 从设备桥接模块具有实际意义。

查重 40%

3.2 功能需求分析

根据项目报告, 项目实现的核心功能可以归纳如下。

查重 42%

第一, I2C 主控制功能。主控制器需要支持读、写、连续写、起始条件、重复起始条件和停止条件等操作。对于实际 I2C 通信而言, 连续写和重复起始很重要, 因

为许多外设要求先写入寄存器地址，再通过重复起始发起读操作。如果控制器只支持单字节简单读写，就难以适配复杂外设。因此，项目将命令接口设计为可以表达多种事务类型，增强了通用性。

第二，I2C 从控制功能。从控制器需要响应主机访问，将写入数据转换为内部数据流输出，并将待发送数据通过 I2C 返回给主机。项目报告指出，`i2c_slave.v` 将 I2C 读写映射为 AXI Stream 数据收发，并支持地址匹配与时钟拉伸。这说明其目标不是固定功能的从设备，而是通用从设备协议引擎。

第三，片上总线包装功能。项目提供基于 AXI-Lite 的 I2C 主机包装层和基于 Wishbone 的 I2C 主机包装层，支持 8 位和 16 位从接口。包装层将协议控制器转换为寄存器可访问外设，使 CPU 或控制逻辑能够通过状态寄存器、命令寄存器、数据寄存器和分频寄存器进行控制。

第四，I2C 从设备桥接功能。项目提供 `i2c_slave_axil_master.v` 和 `i2c_slave_wbm.v`，可把外部 I2C 请求转换为 AXI-Lite 或 Wishbone 主控访问。这种设计允许外部 MCU 通过 I2C 总线间接访问 FPGA 内部寄存器或存储空间，适合低速控制、调试和状态读取场景。

第五，上电初始化功能。项目提供 `i2c_init.v`，通过 ROM 指令表自动驱动一系列 I2C 命令和数据输出，适合在系统启动时配置时钟芯片、PLL、抖动清理器、复用器或其他外设。相比由 CPU 软件完成初始化，硬件初始化状态机可以在无处理器系统中独立工作。

第六，示例与测试功能。项目提供 `i2c_single_reg.v` 作为简化单寄存器 I2C 从设备示例，并提供 MyHDL 测试平台，用于验证主机、从机、初始化模块和总线包装层等功能。

从毕业设计的角度看，上述功能需求之间并不是彼此孤立的。I2C 主控制器解决 FPGA 主动访问外部器件的问题，I2C 从控制器解决 FPGA 被外部控制器访问的问题，片上总线包装层解决 FPGA 内部控制逻辑或软核处理器如何统一管理 I2C 访问的问题，初始化模块解决无处理器或低软件参与场景下的自动配置问题，测试平台则为这些模块能否正确工作提供验证支撑。因此，该项目并非单一协议状态机，而是围绕 I2C 通信形成了较完整的接口组件集合。

在需求优先级上，核心主从控制器应处于最高层级。如果主从控制器无法稳定产生或响应 I2C 时序，其他包装层和桥接层都无法发挥作用。其次是总线适配层，因为在实际 FPGA 系统中，外设接口往往需要通过片上总线进行管理。再次是初始化与示例模块，它们提升了项目在具体场景中的易用性。最后是测试平台，虽然测试平台不直接参与硬件部署，但它决定了项目是否具备可验证、可维护和可扩展的工程基础。

查重 44%

3.3 非功能需求分析

除功能需求外，项目还体现出多项非功能需求。

第一，可移植性。项目使用 Verilog 2001 编写，便于在常见 FPGA 工具链中复用。相较于依赖特定厂商 IP 的方案，纯 Verilog 组件更易迁移到不同 FPGA 平台。但实际迁移仍需要根据目标器件补充约束、顶层封装和板级验证。

第二，可配置性。项目包含分频参数、输入滤波长度、FIFO 深度、数据宽度、地址宽度和设备地址掩码等配置项。可配置性使同一模块能够适配不同 I2C 速率、不同片上总线位宽和不同从设备地址范围。

第三，接口规范性。项目使用 AXI Stream、AXI-Lite 和 Wishbone 等常见接口风格，而不是自定义孤立接口。这有助于与现有 FPGA 系统、软核 CPU 和测试模型集成。

第四，可验证性。项目包含 MyHDL 协同仿真环境，说明其设计目标不仅是提供 RTL 文件，还包括提供验证支撑。对于接口协议组件而言，测试平台是评估可靠性的的重要依据。

第五，安全和约束意识。项目文档明确说明 I2C 引脚采用开漏或三态连接方式，不能将 `scl_o` 直接连接到 `scl_i`。这一点非常重要，因为 I2C 的物理接口模型与普通推挽输出不同。如果连接方式错误，仿真和硬件行为都可能出现问题。

3.4 设计约束与问题

项目报告中还指出了若干约束和已知问题。首先，项目测试依赖 MyHDL、Icarus Verilog 和 `myhdl.vpi` 协同仿真支持。如果用户环境未正确安装这些依赖，就无法直接运行测试脚本。其次，AXI-Lite 和 Wishbone 封装对位宽有参数化约束，源码中包含宽度合法性断言，说明设计对总线宽度组合有一定限制。再次，README 明确说明该仓库已弃用，后续不再维护，新特性与修复迁移到其他仓库。因此，若将其用于新项目，需要评估后继仓库或自行维护风险。

此外，当前项目报告未提供综合结果、时序收敛报告、资源利用率报告、特定 FPGA 板卡顶层工程和约束文件，也未提供实际运行测试的日志。因此，论文在描述验证结果时只能说明“项目具有测试平台和测试设计”，不能直接声称“所有测试已通过”或“硬件验证成功”。

3.5 可行性分析

从技术可行性看，该项目采用 Verilog 2001 编写，并围绕 I2C 这种成熟低速总线协议展开，技术路线明确。I2C 协议本身具有公开资料和大量应用案例，FPGA 工具链也普遍支持 Verilog RTL 综合，因此核心协议控制器在技术上具备可实现基础。项目中使用的 AXI Stream、AXI-Lite 和 Wishbone 接口均是常见数字系统接口形式，相关设计思想成熟，便于与已有系统连接。

从工程可行性看，项目已形成较完整目录结构和模块集合。`rtl/` 目录包含核心可综合代码，`tb/` 目录包含测试模型和测试脚本，README 对项目用途和测试依赖进行了说明。这说明项目已经具备被阅读、仿真和二次移植的基本条件。对于本科毕业设计而言，即使不进行完整板级部署，也可以围绕模块结构、接口关系、仿真测试和工程复用价值展开较完整研究。

从验证可行性看，项目采用 MyHDL 与 Icarus Verilog 的协同仿真方式。Icarus Verilog 是常用开源 Verilog 仿真工具，MyHDL 可用 Python 编写行为模型和测试激励，二者结合可以降低复杂总线模型的编写难度。项目报告已经指出测试脚本不仅进行编译检查，还包含对命令输出、地址序列、数据内容和结束条件的断言。因此，只要环境依赖配置正确，后续具备进一步运行测试并补充实验结果的条件。

从风险角度看，该项目最大的限制是仓库已弃用且缺少实际运行结果。对于毕业论文写作，这要求本文在表述中区分项目已有内容与后续待验证内容；对于实际工程使用，则要求使用者评估后继仓库或自行补充维护。总体而言，该项目适合作为 HDL 接口组件库分析、学习和二次开发基础，作为直接量产级 IP 使用则仍需进一步验证。

4 系统总体设计

4.1 总体架构

从工程组织上看，`verilog-i2c-master` 可以按核心协议、总线扩展、应用示例、公共支撑和测试验证几个层次来理解。

其中，`i2c_master.v` 和 `i2c_slave.v` 构成最基础的协议层，分别处理 I2C 主机和从机行为。它们直接面对时序、地址、读写方向、应答以及状态反馈等问题，是后续包装层和桥接模块能够工作的前提。

扩展层由 `i2c_master_axil.v`、`i2c_master_wbs_8.v`、`i2c_master_wbs_16.v`、`i2c_slave_axil_master.v` 和 `i2c_slave_wbm.v` 构成。该层将核心协议能力与片上总线系统连接起来。一方面，主机包装层把 I2C 主控制器暴露为 AXI-Lite 或 Wishbone 从设备；另一方面，从机桥接层把外部 I2C 主设备的访问转换为内部 AXI-Lite 或 Wishbone 主控事务。

应用层由 `i2c_init.v` 和 `i2c_single_reg.v` 构成。前者提供上电初始化状态机模板，可用于批量配置外设；后者提供简化单寄存器从设备示例，适合教学和快速验证。

支撑层包括 `axis_fifo.v`。该模块是通用 AXI4-Stream FIFO，用于数据缓冲和流式接口解耦。通过 FIFO，命令和数据可以在协议状态机和上层接口之间平滑传递，降低时序耦合。

验证层位于 `tb/` 目录，包含 AXI-Lite、AXI Stream、I2C、Wishbone 等行为模型以及多个测试脚本。验证层使项目能够在仿真环境中构造主从交互、数据读写和初始化序列等测试场景。

4.2 工程目录结构

项目报告给出的工程结构如下：

```
verilog-i2c-master/
  README.md
  rtl/
    axis_fifo.v
    i2c_init.v
  □ □ □ i2c_master.v
    i2c_master_axil.v
    i2c_master_wbs_8.v
  □ □ □ i2c_master_wbs_16.v
    i2c_single_reg.v
```

```

    i2c_slave.v
  □ □ □ i2c_slave_axil_master.v
        i2c_slave_wbm.v
  tb/
    axil.py
    axis_ep.py
  □ □ i2c.py
    wb.py
    test_i2c*.py
    test_i2c*.v

```

从该目录结构可以看出，项目将可综合 RTL 文件集中放置在 `rtl/` 目录，将测试平台文件集中放置在 `tb/` 目录。这种结构符合开源 HDL 项目的常见组织方式，有利于用户快速定位设计文件和验证文件。`README.md` 则提供模块说明、接口信息和测试方式，是用户理解项目的入口。

这种目录安排也体现了 HDL 工程中常见的设计与验证分离思路。可综合 RTL 和测试代码的用途并不相同，前者最终会进入综合、布局布线流程并映射到 FPGA 资源，后者主要用于仿真，不会成为硬件实现的一部分。把两类文件分开放置，可以减少误用，也方便脚本分别处理设计文件和测试文件。另外，项目没有把全部逻辑集中在一个大模块中，而是把主机、从机、包装层、桥接层和示例模块拆成多个文件，用户可以根据自己的系统需求选择集成。

从复用角度看，`rtl/` 中每个模块都对对应较明确的应用场景。用户如果只需要最基础的 I2C 主机，可以直接集成 `i2c_master.v`；如果已有 AXI-Lite 控制总线，则可以选择 `i2c_master_axil.v`；如果系统使用 Wishbone，则可以选择对应 Wishbone 版本；如果需要外部 MCU 通过 I2C 访问 FPGA 内部寄存器，则可以使用从机桥接模块。这种按功能拆分的组织形式降低了项目集成成本。

4.3 数据流设计

4.3.1 主机数据流 I2C 主机侧的数据流可以概括为：上层逻辑写入命令和数据，I2C 主控制器解析命令并驱动总线，读回数据再返回上层逻辑。具体过程包括以下步骤。

首先，上层逻辑通过 AXI Stream 或寄存器接口提交命令。命令可以表示起始、读、写、连续写或停止等操作。如果使用 AXI-Lite 或 Wishbone 包装层，则命令由 CPU 或控制器写入命令寄存器；如果直接使用核心模块，则命令通过流式接口输入。

其次，控制器将命令转换为 I2C 总线时序。内部命令状态机负责决定当前执行地址阶段、读阶段、写阶段还是停止阶段；底层 PHY 状态机负责在 SCL 和 SDA 上产生符合协议的位级变化。

再次，若执行写操作，待发送数据通过写数据通道进入控制器，并在 SCL 时钟控制下逐位输出；若执行读操作，从设备返回的数据被采样并通过输出数据通道返回。

最后，控制器通过状态信号反馈执行情况。例如，当从设备未应答时 `missed_ack`

可用于提示上层逻辑通信失败。若总线仍处于连续访问状态，相关状态也可反映当前总线是否忙碌或保持控制。

4.3.2 从机数据流 I2C 从机侧的数据流以外部主设备发起访问为起点。外部主设备首先发送从设备地址和读写位，从控制器检测总线并判断地址是否匹配。如果地址匹配，从控制器返回 ACK 并进入读或写事务。

在写事务中，外部主设备发送的数据字节被从控制器接收，并转换为内部 AXI Stream 输出。这样，上层逻辑可以通过流式接口读取外部写入的数据。在读事务中，从控制器从 AXI Stream 输入端获取待发送数据，并在 I2C 主设备提供的时钟下发送给主设备。如果待发送数据尚未准备好，从控制器可以通过时钟拉伸保持 SCL 为低，从而等待内部数据就绪。

该设计使 I2C 从控制器不绑定具体寄存器模型，而是提供通用数据收发能力。用户可以在其外部连接寄存器文件、FIFO、片上总线桥接逻辑或其他控制模块。

4.3.3 I2C 到片上总线桥接数据流 项目提供的从机桥接层进一步扩展了 I2C 从设备能力。以 `i2c_slave_axil_master.v` 和 `i2c_slave_wbm.v` 为例，外部 MCU 可以通过 I2C 写入目标地址，然后继续写入数据或发起读操作。模块在内部保存地址指针，并将后续事务转换为 AXI-Lite 或 Wishbone 总线访问。

这种桥接流程可以用于低速调试和控制。例如，外部 查重 42% 需要直接连接 FPGA 内部总线，只需通过 I2C 即可访问部分寄存器空间。由于地址指针支持自动递增，连续寄存器读写也更方便。对于数据宽度超过 8 位的片上总线，桥接模块还需要处理对齐、补零、合并和拆分等问题，项目报告指出 AXI-Lite 和 Wishbone 桥接模块均包含类似处理逻辑。

4.4 控制流设计

项目的控制流核心是状态机。查重 47% **I2C 主控制器内部可理解为两级状态机：**较高层的命令状态机负责处理事务级操作，如发起开始条件、发送地址、执行读写、生成停止条件等；较底层的 PHY 状态机负责位级时序，如 SCL 拉低、SDA 设置、SCL 拉高、数据采样和 ACK 检测等。两级状态机分离有利于降低设计复杂度，使事务控制和物理时序控制职责清晰。

I2C 从控制器的控制流则以总线监听为起点。模块需要检测起始条件，接收地址，判断地址匹配，返回应答，然后根据读写位进入接收或发送状态。由于从设备由外部主机提供时钟，它还需要在输入数据、输出数据和时钟拉伸之间保持协调。

初始化模块 `i2c_init.v` 的控制流则基于 ROM 指令表。模块在 `start` 拉起后，按顺序读取初始化指令，将其转换为 I2C 主控命令和数据流。指令表可包含写入数据和延时命令，从而描述多设备、多步骤的初始化过程。这种方法把固定初始化流程从复杂状态机中抽象为指令序列，提高了可配置性和可读性。

4.5 接口设计原则

该项目的接口设计体现出“协议核心简单化、系统集成标准化”的原则。核心 I2C 模块采用接近协议语义的信号和 AXI Stream 风格数据通道，使其既能被上层

包装层驱动，也能直接被自定义逻辑使用。包装层则采用 AXI-Lite 和 Wishbone 等标准片上总线接口，使控制器能够被软核 CPU 或系统控制器统一管理。

I2C 物理接口采用 `scl_i`、`scl_o`、`scl_t` 和 `sda_i`、`sda_o`、`sda_t` 的形式。这种输入、输出、三态控制分离的接口形式适合 FPGA I/O 单元建模，也符合 I2C 开漏连接特点。由于 I2C 总线高电平通常由上拉电阻提供，设备只主动拉低总线或释放总线，因此不能像普通推挽信号一样简单将输出反馈到输入。项目报告中特别指出不能将 `scl_o` 直接连接到 `scl_i`，说明设计者对物理层连接约束有明确提示。

在内部数据接口上，AXI Stream 风格握手可以提高模块之间的解耦程度。I2C 总线速率通常远低于系统时钟，如果上层逻辑必须等待每一个 I2C 位级动作完成，会造成控制复杂且效率较低。通过命令流、写数据流和读数据流，上层逻辑可以以事务为单位提交操作，协议模块则在内部逐步执行。这种设计也为 FIFO 缓冲和寄存器包装提供了基础。

在片上总线接口上，AXI-Lite 和 Wishbone 的引入使项目可以适配不同系统生态。AXI-Lite 适合较新的 SoC 和 FPGA IP 集成环境，Wishbone 则在开源软核和传统开源 HDL 项目中常见。项目同时提供两类接口，增强了组件库的适用范围。

5 详细设计与实现分析

5.1 I2C 主控制器设计

`i2c_master.v` 是项目中最关键的文件之一，主要负责 I2C 主模式访问。根据项目说明，它的命令接口能够表达 `read`、`write`、`write multiple`、`stop` 等操作；在事务控制上，既支持显式开始和停止，也能结合总线状态自动发起必要的开始过程。模块还通过 `prescale` 设置 I2C 时钟分频，并用 `missed_ack` 向上层反馈未收到应答的情况。整体上看，它内部把事务级命令控制和底层 PHY 时序控制分开处理。

主控制器的难点在于把上层比较抽象的事务命令，稳定地转换成 SDA 和 SCL 上的具体时序。对上层逻辑来说，它更希望表达“读一个字节”“连续写多个字节”或“结束本次访问”这类动作；但在 I2C 总线上，这些动作最终都要落实到逐位驱动、逐位采样以及 ACK 判断。把状态机分成事务控制和物理时序两个层次，可以让设计更清楚：前者决定当前要发地址、写数据、读数据还是停止，后者负责在合适的时刻改变或采样总线信号。

`prescale` 分频参数用于生成 I2C 时钟。项目报告指出源码中给出公式 $\text{prescale} = \text{Fclk} / (\text{FI2Cclk} * 4)$ 。这说明模块内部可能将一个 I2C 时钟周期划分为若干阶段，用于控制 SCL 低电平、高电平以及数据建立和保持时间。通过分频参数，用户可以在不同系统时钟频率下生成目标 I2C 速率。

查重 41%

`missed_ack` 是 I2C 主机设计中的重要状态。当主设备发送地址或数据后，从设备应在第九个时钟周期拉低 SDA 表示应答。如果未检测到应答，控制器需要向上层报告异常。该状态可以帮助软件或控制逻辑判断外设是否存在、总线是否正常、地址是否正确或设备是否忙碌。

5.2 I2C 从控制器设计

`i2c_slave.v` 对应 I2C 从机功能。项目资料显示, 该模块能够解析主机发起的读写事务, 把外部写入的字节送到 AXI Stream 输出端, 也可以从 AXI Stream 输入端取出待发送数据并返回给主机。当内部数据暂时没有准备好时, 模块还可以通过时钟拉伸进行等待。除此之外, 它提供 `busy`、`bus_address`、`bus_addressed`、`bus_active` 等状态信号, 并通过 `device_address` 与 `device_address_mask` 完成地址匹配配置。

查重 60%

I2C 从控制器的设计与主控制器不同。主控制器主动产生时钟和事务, 而从控制器必须被动监听总线。它需要在检测到起始条件后开始接收地址位, 并根据设备地址和掩码判断是否响应。地址掩码机制有利于支持多个地址或地址范围匹配, 使设计更灵活。

在写事务中, 从控制器接收外部主机发送的数据字节, 并通过输出 AXI Stream 通道交给内部逻辑。在读事务中, 从控制器需要从输入 AXI Stream 通道获得待发送数据。如果上层数据尚未准备好, 模块可以通过拉低 SCL 实现时钟拉伸。时钟拉伸是 I2C 协议允许从设备暂停主机时钟的一种机制, 对于内部处理速度慢于总线访问速度的设备非常有用。

状态信号的设计也很重要。`busy` 可表示模块正在处理事务, `bus_active` 可表示总线处于活动状态, `bus_addressed` 可表示当前从设备被主机选中, `bus_address` 可输出当前地址信息。这些状态有助于上层逻辑进行调试、统计和控制。

5.3 AXI-Lite 主机包装层设计

`i2c_master_axil.v` 将 I2C 主控制器封装为 AXI-Lite 从设备。根据项目报告, 该模块提供 4 个主要寄存器: 状态、命令、数据、分频。状态寄存器包含 `busy`、`bus_cont`、`bus_act`、`miss_ack` 以及 FIFO 状态; 命令寄存器允许组合 `start`、`read`、`write`、`write` 查重 51%、`tipple` 和 `stop`; 数据寄存器既可写入待发送字节, 也可读取接收字节; 模块还提供可配置的命令 FIFO、写 FIFO 和读 FIFO 深度。

AXI-Lite 包装层的主要作用, 是让 I2C 控制器能够像普通外设一样被 CPU 或软核处理器访问。在 SoC 风格的 FPGA 系统中, 软件通常通过寄存器读写来配置外设和读取状态。如果模块只暴露 AXI Stream 命令接口, 软件控制会不够直接; 加入 AXI-Lite 包装层后, 控制流程就可以转化为标准总线上的寄存器访问, 集成方式也更接近常见外设 IP。

加入 FIFO 后, 软件侧和 I2C 协议执行侧之间不必完全同步。软件可以先写入若干命令或数据, 控制器再按照 I2C 时序逐步处理; 读回的数据也可以先缓存起来, 等待软件读取。这样能够减少软件频繁等待底层总线动作完成的情况。由于 I2C 本身速率较低, FIFO 不一定要设置得很深, 但可配置深度可以让模块适应不同控制节奏。

5.4 Wishbone 主机包装层设计

项目提供 `i2c_master_wbs_8.v` 和 `i2c_master_wbs_16.v` 两个 Wishbone 主机包装层。根据项目报告, `i2c_master_wbs_8.v` 将状态、FIFO 状态、命令地址、命令、数据和分频映射到不同地址, 适合基于 Wishbone 的软核系统或开源总线

架构。`i2c_master_wbs_16.v` 可以合理理解为采用相近思路，但接口和寄存器宽度适配为 16 位。

Wishbone 总线在开源 FPGA 项目中具有较长历史，许多软核和外设 IP 使用该接口。项目同时提供 AXI-Lite 和 Wishbone 封装，说明其目标用户不局限于某一种片上总线生态。对于组件库而言，多总线适配可以扩大复用范围，也有助于用户在已有系统中直接集成。

8 位和 16 位版本的区别可能体现在数据总线宽度、地址映射和寄存器访问方式上。由于 I2C 数据本身以字节为基本单位，8 位接口具有自然匹配优势；16 位接口则更适合某些软核或系统总线配置。无论哪种版本，包装层的核心任务都是把总线读写转换为 I2C 控制器可理解的命令、数据和配置操作。

5.5 I2C 从设备桥接层设计

`i2c_slave_axil_master.v` 和 `i2c_slave_wbm.v` 是项目中较有特色的模块。它们不是简单的 I2C 从设备，而是将外部 I2C 主机访问桥接到 FPGA 内部片上总线。根据项目报告，`i2c_slave_axil_master.v` 中，I2C 写操作先访问内部地址寄存器，再写 AXI-Lite 地址空间；I2C 读操作从当前内部地址开始发起 AXI-Lite 读；对于宽于 8 位的总线，模块处理对齐、补零和同地址数据合并；地址指针会自动递增。

通过这类桥接设计，FPGA 可以作为一个 I2C 外设接到外部控制器上。外部 MCU 只要通过 I2C 写入地址和数据，就能够间接访问 FPGA 内部的部分寄存器空间。这对系统调试、板级管理、状态读取和低速控制比较有用。比如某个 FPGA 系统内部有多个控制寄存器，但外部主控只预留了 I2C 接口，此时桥接模块就能在不增加复杂外部总线的情况下提供控制通道。

`i2c_slave_wbm.v` 与 AXI-Lite 版本类似，但桥接到 Wishbone 主接口。项目报告指出，该模块保留了参数化数据宽度、地址宽度、选择信号宽度，对位宽和字宽进行合法性断言，并具有地址寄存器、自动递增和读写合并语义。这说明桥接模块不仅实现协议转换，还考虑了不同总线数据宽度带来的访问粒度问题。

5.6 初始化模块设计

`i2c_init.v` 可以看作一个用于上电配置的模板化状态机。项目材料表明，它把初始化步骤写入 `init_data` ROM，指令宽度为 9 位，并支持单设备和多设备两类初始化流程，也可以在序列中插入延时。`start` 信号有效后，模块会按照表中内容依次输出 I2C 主控命令和数据流，从而完成预配置过程。

在实际硬件系统里，很多外设上电后都需要写入一组配置寄存器。时钟芯片可能要设置输出频率和 PLL 参数，图像传感器可能要配置分辨率、曝光和数据格式，电源管理芯片也常常需要配置电压轨和启动时序。如果系统中没有嵌入式 CPU，或者某些外设必须在 CPU 正式工作前完成配置，硬件初始化状态机就能发挥作用。

将初始化流程编码为 ROM 指令表具有明显优势。第一，初始化数据与状态机控制逻辑分离，便于修改配置而不改动核心控制流程。第二，多个设备初始化可以按表顺序执行，结构清晰。第三，延时命令可以处理某些外设需要等待稳定时间

的场景。第四，模块输出 I2C 主控命令与数据流，可复用主控制器完成实际总线时序。

5.7 单寄存器从设备示例

`i2c_single_reg.v` 是简化的 I2C 从设备实现。项目报告指出，该模块固定设备地址 `DEV_ADDR`，只提供单字节寄存器读写，接口简单，适合教学、快速验证或构建最小化外设示例。

虽然该模块功能简单，但在组件库中具有示例价值。对于初学者而言，直接理解通用从控制器和片上总线桥接模块可能较难；单寄存器从设备则提供了一个最小可理解实例，展示 I2C 地址匹配、写入寄存器和读回寄存器的基本流程。对于工程验证而言，该模块也可作为测试主控制器读写行为的简单目标。

5.8 AXI Stream FIFO 设计

`axis_fifo.v` 是通用 AXI4-Stream FIFO。项目报告指出，该模块参数化深度、数据宽度、`tkeep`、`tlast`、`tuser` 等，支持帧模式和坏帧丢弃机制，在 I2C 项目中主要承担缓冲和解耦作用。

在 I2C 控制器中，协议时序通常比片上总线或内部逻辑慢得多。如果上层逻辑和 I2C 状态机直接耦合，可能导致接口控制复杂。通过 FIFO，命令、写数据和读数据可以缓存起来，使生产者和消费者在一定范围内独立运行。对于 AXI-Lite 包装层，FIFO 还能减少软件访问时序与 I2C 执行时序之间的直接依赖。

从工程角度说，FIFO 不只是用来临时存数据，它还可以缓冲不同模块之间的速度差。AXI-Lite 或 Wishbone 一侧通常运行在系统时钟下，写命令的速度可能比 I2C 总线真正执行事务的速度快得多；而 I2C 又受到分频设置和协议时序约束，不能无限加快。如果没有缓冲，上层逻辑就需要经常等待底层事务完成，软件控制体验和系统吞吐都会受到影响。配置合适深度的 FIFO 后，模块可以先接收一部分命令或数据，再由协议状态机按顺序处理。

此外，FIFO 还有助于提高模块边界清晰度。上层包装层只需要关心 FIFO 是否满、是否空以及何时读写数据，而底层 I2C 只需要按照握手信号获取命令和数据。两者之间不必共享复杂内部状态。这种边界清晰的设计方式符合可复用 IP 的基本要求。

5.9 关键参数与配置分析

项目报告列出了多个关键参数，包括 `prescale`、`FILTER_LEN`、`CMD_FIFO_DEPTH`、`WRITE_FIFO_DEPTH`、`READ_FIFO_DEPTH`、`DATA_WIDTH`、`ADDR_WIDTH`、`STRB_WIDTH`、`WB_DATA_WIDTH`、`WB_ADDR_WIDTH`、`WB_SELECT_WIDTH`、`device_address` 和 `device_address_mask` 等。这些参数覆盖了时钟、滤波、缓冲、片上总线位宽和从设备地址匹配等多个方面。

`prescale` 是 I2C 主控制器中最直接影响通信速率的参数。由于 FPGA 系统时钟通常远高于 I2C 总线速率，控制器需要通过分频产生合适的 SCL 时序。项目报告中给出的公式表明，设计者将 I2C 一个周期划分为多个内部阶段，以便控制数

据建立、采样和保持过程。用户在移植模块时，应根据系统时钟频率和目标 I2C 速率重新计算该参数。

`FILTER_LEN` 与输入滤波有关。I2C 总线可能受到毛刺、上升沿缓慢或外部干扰影响，输入滤波可以提高信号判断稳定性。滤波长度过短可能无法过滤噪声，过长则可能影响边沿响应速度，因此应结合目标总线速率和硬件环境进行配置。

查重 53%

`FIFO` 深度参数决定了命令和数据缓冲能力。较大的 `FIFO` 可以容纳更多未执行命令或待读数据，提高突发访问场景下的缓冲能力，但也会增加资源消耗。对于低速控制场景，较小 `FIFO` 可能已经足够；对于需要连续读写多个寄存器的外设配置场景，适当增加 `FIFO` 深度可以减少上层等待。

总线位宽参数影响 AXI-Lite 和 Wishbone 包装层的地址映射和数据对齐。对于 I2C 这种以字节为基本传输单位的协议，当片上总线宽度大于 8 位时，桥接模块需要处理字节选择、地址递增、数据合并和补零等问题。项目报告指出相关模块中存在位宽合法性断言，说明设计对参数组合进行了一定约束。

`device_address` 和 `device_address_mask` 则用于 I2C 从设备地址匹配。通过掩码机制，一个模块可以响应特定地址或地址范围，增强配置灵活性。但在实际系统中，地址配置必须避免与总线上其他设备冲突，否则可能出现多个从设备同时应答的问题。

5.10 实现小结

综合上述分析，`verilog-i2c-master` 的实现思路可以概括为：以 I2C 主从协议状态机为核心，以 AXI Stream 风格接口实现命令与数据解耦，以 AXI-Lite 和 Wishbone 包装层面向系统集成，以初始化和单寄存器示例面向具体应用，以测试平台支撑功能验证。该实现方式既体现了 HDL 模块化设计思想，也体现了开源接口组件库的工程组织方式。

需要说明的是，本文对实现部分的分析主要来自项目报告和已有材料，并没有对全部源码逐行展开审查。因此，具体状态机编码、寄存器位定义以及边界条件处理，还需要在后续工作中结合源码细读、仿真波形和测试日志进一步确认。

6 测试与验证分析

6.1 测试平台组成

项目报告显示，`verilog-i2c-master` 包含较完整的测试体系。`tb/axil.py` 提供 AXI4-Lite 主机与内存行为模型；`tb/axis_ep.py` 提供 AXI Stream 端点模型；`tb/i2c.py` 提供 I2C 主从模型；`tb/wb.py` 提供 Wishbone 主机与 RAM 模型；多个 `test_*.py` 对应不同 RTL 模块的专项测试。

这种测试安排体现出比较明显的分层验证思路。测试主控制器时，可以把 I2C 存储器模型作为访问对象，观察不同地址、延迟和命令数据组合下的行为；测试从控制器时，可以由 I2C 主机模型驱动 DUT，再检查地址响应、数据接收、数据发送和时钟拉伸是否符合预期；测试初始化模块时，则重点看输出的地址和数据序列是否与表中配置一致；对于总线包装层，可以借助 AXI-Lite 或 Wishbone 行为模型模拟软件访问或片上总线访问过程。

6.2 协同仿真流程

根据项目报告，项目测试依赖 MyHDL、Icarus Verilog 和 myhdl.vpi。典型流程包括安装 MyHDL、安装 Icarus Verilog、确保 myhdl.vpi 正确安装，然后使用 `py.test`、`nose` 或直接运行 Python 测试脚本。测试脚本内部会调用类似 `iverilog -o test_xxx.vvp ...` 的命令编译 Verilog 文件，再使用 `vvp -m myhdl` 运行协同仿真。

该流程中，Python 测试脚本既负责生成激励，也负责检查结果。Verilog 被测模块作为 DUT 参与仿真。MyHDL 提供了 Python 与 Verilog 仿真之间的连接机制，使 Python 行为模型能够与 HDL 信号交互。相较于手写纯 Verilog testbench，这种方法更容易构造复杂行为模型，例如 I2C 存储器、AXI-Lite 内存和 Wishbone RAM。

6.3 验证覆盖内容

从项目报告可知，测试脚本覆盖了多个方面。`test_i2c_master.py` 构造了两个 I2C 存储器模型，并测试不同地址、延迟和命令/数据流；`test_i2c_slave.py` 使用 I2C 主机模型驱动 DUT，并通过总线线与逻辑模拟开漏连接；`test_i2c_init.py` 检查初始化模块发出的地址与数据序列是否符合预期。

这些测试内容说明项目验证并非只检查编译是否通过，而是针对协议行为进行功能验证。对于 I2C 主机，测试需要关注是否正确产生地址、读写位、数据字节和停止条件，是否能够接收数据并检测应答。对于 I2C 从机，测试需要关注地址匹配、数据接收、数据发送和时钟拉伸。对于初始化模块，测试需要关注指令表是否被正确解释，输出序列是否符合预设。

6.4 验证结果与谨慎表述

这里需要特别说明，本文依据的是项目报告和已有文件材料，而材料中并没有给出本次实际运行仿真的结果。因此，论文不能直接写成“测试已经全部通过”。比较稳妥的表述是：项目作者为主要模块准备了相应测试，测试结构中包含对命令输出、地址序列、数据内容和结束条件的检查，验证设计覆盖了主机、从机、初始化流程以及部分总线桥接场景。

也就是说，该项目确实具备较完整的仿真测试框架，但现有论文材料还不足以证明这些测试已经在本地环境中成功执行。若后续要让毕业论文成果更加完整，应搭建 MyHDL、Icarus Verilog 和 myhdl.vpi 等依赖环境，实际运行测试脚本，并保存日志、波形截图或断言结果，再把这些内容补入第六章作为实验依据。

6.5 测试不足与改进方向

从项目报告看，当前材料未提供综合结果、时序收敛报告和资源利用率数据，也未提供特定 FPGA 板卡的顶层工程和约束文件。因此，该项目验证仍主要停留在仿真层面。仿真可以证明功能行为在测试场景下符合预期，但不能替代综合后时序分析和硬件上板验证。

后续可以从以下方向补充。第一，运行现有 MyHDL 测试，记录测试命令、通过结果和关键波形。第二，选择一个目标 FPGA 器件进行综合，记录 LUT、触发器、块 RAM 等资源使用情况。第三，在不同 I2C 分频参数下进行时序分析，确

认最高可运行频率和约束满足情况。第四，构建板级示例工程，连接真实 I2C 外设进行读写验证。第五，引入更现代的 cocotb 或 SystemVerilog 验证方法，提高测试可维护性和覆盖率。

7 总结与展望

7.1 全文总结

本文基于 `project-report.md` 对 `verilog-i2c-master` 项目进行了本科毕业论文形式的整理和分析。该项目是一个面向FPGA/HDL 开发的 I2C 接口组件库，提供 I2C 主从控制器、AXI-Lite 和 Wishbone 包装层、I2C 到片上总线桥接模块、初始化状态机、单寄存器从设备示例以及 MyHDL 协同仿真测试平台。

从需求分析部分可以看出，该项目主要面向 FPGA 外设控制、上电初始化、片上总线桥接和寄存器访问等场景，功能覆盖面较完整，非功能层面也体现了可移植、可配置、接口规范和便于验证等特点。从总体设计部分看，项目采用分层结构，把协议核心、总线适配、应用示例和测试支撑分开组织，这有助于后续复用和系统集成。从详细设计部分看，主控制器把事务级命令状态机和位级 PHY 状态机分离，从控制器通过 AXI Stream 与上层逻辑解耦，而总线包装层和桥接层则让 I2C 协议能力可以接入更复杂的片上系统。

在测试与验证方面，项目提供了较完整的 MyHDL 协同仿真环境，包括 AXI-Lite、AXI Stream、I2C 和 Wishbone 行为模型。测试脚本针对主机、从机和初始化模块设计了功能验证场景。由于当前材料未实际执行仿真，本文仅将其表述为“具备测试平台和验证设计”，不将测试通过作为事实结论。

7.2 项目优势

结合前文分析，该项目的优势可以概括为以下几个方面。

一是模块边界比较清楚。核心协议模块、总线包装模块、应用示例模块和测试平台各自承担不同任务，阅读和复用时都比较容易定位。

二是接口适配能力较强。项目不仅包含基础 I2C 主从控制器，还提供 AXI-Lite 和 Wishbone 相关适配层，可以接入不同类型的片上系统。

三是参数配置较丰富。分频、滤波长度、FIFO 深度、数据位宽、地址位宽和设备地址掩码等内容都可以调整，这对不同系统中的移植比较有帮助。

四是验证支撑相对完整。项目提供 MyHDL 协同仿真测试平台，测试文件并不是简单占位，而是围绕主机、从机和初始化模块设计了具体场景。

五是具有一定教学参考价值。项目展示了从基础协议控制器到系统级总线接口封装的过程，适合用来学习 HDL 模块化设计和接口组件库组织方法。

7.3 项目不足

项目也存在一些不足。首先，README 已明确说明该仓库已弃用，后续不再维护。这意味着在新项目中直接使用该仓库需要承担维护风险。其次，当前材料中缺少综合报告、时序报告、资源利用率报告和 FPGA 板级验证结果，因此无法评

估该设计在具体器件上的资源开销和时序性能。再次，项目没有提供特定开发板的顶层工程、引脚约束和真实外设演示，用户需要自行完成上板集成。最后，测试平台依赖 MyHDL 和 Icarus Verilog，对初学者而言可能存在环境搭建门槛。

7.4 后续展望

后续工作可以围绕工程化、验证完善和文档增强三个方向展开。

在工程化方面，可以选择常见 FPGA 开发板构建示例工程，补充约束文件、顶层连接和外设读写演示。也可以根据目标器件生成综合报告和时序报告，比较不同 FIFO 深度、不同总线包装层和不同分频参数下的资源使用情况。

在验证完善方面，可以实际运行现有测试脚本，保存测试日志和波形文件；也可以引入 cocotb 或 SystemVerilog 验证框架，构建更现代的回归测试环境。对于 I2C 协议，还可以增加异常场景测试，例如从设备无应答、总线忙、时钟拉伸超时、连续读写边界和不同地址掩码匹配。

在文档完善方面，后续可以增加模块接口时序图、寄存器映射表、典型使用流程和移植说明。AXI-Lite 与 Wishbone 包装层最好配套给出软件访问示例，i2c_init.v 也可以补充初始化指令表的配置方法和外设配置案例，这样更便于使用者理解和迁移。

总体来看，verilog-i2c-master 虽然已经停止维护，但它在模块结构、接口封装和测试组织上的做法仍然有参考意义。对该项目进行系统整理，可以帮助理解 FPGA I2C 接口组件库的设计思路，也能为后续构建更完整、更现代的 HDL 外设接口库提供一定基础。

致谢

在本文整理过程中，项目报告 project-report.md 为论文提供了主要事实依据，包括项目目标、模块结构、功能需求、实现内容和测试平台说明。感谢开源项目作者提供 Verilog I2C 接口组件库，使学习者能够通过真实工程案例理解 I2C 协议控制器、片上总线封装和协同仿真测试方法。感谢指导教师 in 毕业设计选题、论文结构和技术表达方面给予的指导。由于当前材料未包含实际仿真运行日志、综合报告和板级测试结果，本文在相关结论中采用谨慎表述，后续仍需通过实际实验进一步完善。

参考文献

注：以下参考文献主要依据项目材料和通用技术来源整理。正式提交论文前，建议进一步通过 Google Scholar、CNKI 或学校图书馆补充 I2C 协议、FPGA 接口设计、AXI/Wishbone 总线和 HDL 验证相关学术文献，并按 GB/T 7714—2015 核对格式。

- [1] Philips Semiconductors. The I2C-bus specification and user manual[S].
- [2] ARM. AMBA AXI and ACE Protocol Specification[S].
- [3] OpenCores. Wishbone B4 WISHBONE System-on-Chip Interconnection Architecture for Portable IP Cores[S].

[4] IEEE. IEEE Standard for Verilog Hardware Description Language[S].
[5] verilog-i2c-master 项目 README 与 RTL 源码材料 [EB/OL].
[6] project-report.md: 项目规划设计与详细内容总结 [Z].

附录 A 项目主要模块清单

模块	功能说明
i2c_master.v	I2C 主控制器，负责主模式读写、起始、停止、ACK 检测和时序控制
i2c_slave.v	I2C 从控制器，负责地址匹配、数据接收、数据发送和时钟拉伸
i2c_master_axil.v	AXI-Lite 从接口包装层，通过寄存器控制 I2C 主机
i2c_master_wbs_8.v	8 位 Wishbone 从接口包装层
i2c_master_wbs_16.v	16 位 Wishbone 从接口包装层
i2c_slave_axil_master.v	I2C 从设备到 AXI-Lite 主接口的桥接模块
i2c_slave_wbm.v	I2C 从设备到 Wishbone 主接口的桥接模块
i2c_init.v	基于 ROM 指令表的 I2C 初始化状态机
i2c_single_reg.v	单字节寄存器型 I2C 从设备示例
axis_fifo.v	AXI Stream FIFO，用于缓冲和接口解耦

附录 B 后续实验建议

为了使论文成果更加完整，建议后续补充以下实验材料：

- 1. 搭建 MyHDL 与 Icarus Verilog 环境，运行项目测试脚本。
- 2. 保存 test_i2c_master.py、test_i2c_slave.py、test_i2c_init.py 等测试日志。
- 3. 对关键事务生成波形截图，展示起始条件、地址阶段、数据阶段、ACK 和停止条件。
- 4. 选择目标 FPGA 器件进行综合，记录 LUT、触发器、块 RAM 等资源占用。
- 5. 添加一个真实 I2C 外设或仿真存储器的演示工程，验证控制器实际读写流程。
- 6. 根据实验结果补充论文第六章，使“验证结果”从结构分析提升为可复现实验结论。